Complemento Práctico: Estructuras y Memoria

Llegamos al final de la unidad con dos conceptos que cambian la forma en que gestionamos los datos. Por un lado, la **recursividad**, que suele parecer "magia" hasta que entendemos qué pasa en la memoria RAM (la Pila). Por otro, los **Arreglos**, que nos permiten dejar de crear variables sueltas (a, b, c) para empezar a manejar colecciones masivas.

En este documento veremos por qué la recursividad puede hacer explotar la memoria y qué "valores fantasma" esconde un Arreglo cuando lo creamos. Estos conceptos son fundamentales para comprender cómo Java administra la memoria durante la ejecución de nuestros programas.

Contenido

Visualización de la Pila

La metáfora de la Pila de Platos (Stack) para entender cómo funciona la memoria de ejecución

Errores Críticos

StackOverflowError vs ArrayIndexOutOfBounds: los dos enemigos más comunes

Valores Fantasma

Los valores por defecto (Default Values) que Java asigna automáticamente a los arreglos

Modularización

Cómo pasar arreglos como parámetros y crear código reutilizable y profesional

La Metáfora de la Pila (The Stack)

En el video explicamos que la memoria de ejecución funciona como una pila de platos. Cada vez que llamamos a un método, Java pone un "plato" (Stack Frame) encima del anterior.

- Si llamamos al método A(), y este llama a B(), el plato B se pone sobre el A
- Cuando B() termina, su plato se retira y volvemos a ver el plato A
- La memoria tiene un límite físico: el "techo" de nuestra pila

El Peligro de la Recursividad

En la recursividad, el método se llama a sí mismo. Si olvidamos el **Caso Base** (el freno), seguimos poniendo platos infinitamente (A llama a A, que llama a A...). Cuando los platos tocan el techo, ocurre el error StackOverflowError (Desbordamiento de Pila) y el programa muere instantáneamente.

Visualizando el Stack Overflow

Llamada Inicial

El programa inicia y llama al método recursivo por primera vez. Se coloca el primer "plato" en la pila.

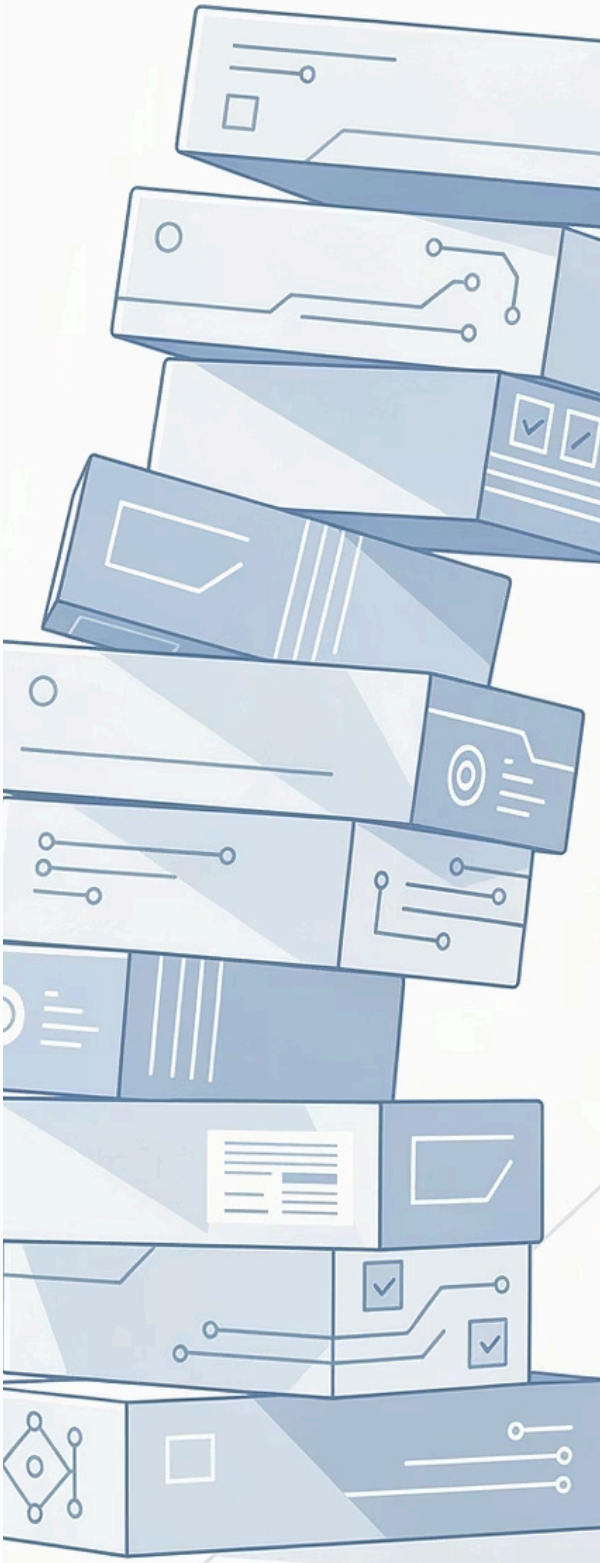
Recursión Sin Freno

El método se llama a sí mismo repetidamente. Cada llamada apila un nuevo frame en memoria sin liberar los anteriores.

Límite Alcanzado

La memoria de la pila se agota. Java lanza StackOverflowError y termina la ejecución abruptamente.

> "La recursividad es como una escalera: necesitás saber cuándo parar de subir para poder volver a bajar. Sin caso base, es una escalera al infinito."



Los "Valores Fantasma" (Default Values)

Una duda común en los videos: *"Profe, creé un arreglo de 10 números pero no le cargué nada. ¿Qué hay adentro?"*

A diferencia de las variables locales (que dan error si no se inicializan), los Arreglos en Java se autocompletan al nacer. Java es generoso y llena automáticamente cada posición con un valor por defecto según el tipo de dato.

Esto evita errores en tiempo de compilación, pero puede generar bugs sutiles si no lo conocés. Un arreglo de enteros lleno de ceros puede parecer "vacío", pero en realidad está **lleno de valores reales** que podés usar y modificar.

int, byte, short

0

Valores numéricos enteros se inicializan en cero

double, float

0.0

Valores numéricos decimales también parten de cero

boolean

false

Los booleanos arrancan en falso por defecto

String / Objetos

null

¡Cuidado! Intentar usar un null causa NullPointerException

El Error "Off-by-One"

El error más clásico al trabajar con Arrays es el ArrayIndexOutOfBoundsException. Ocurre cuando intentamos acceder a una posición que no existe. Es como intentar abrir el cajón número 6 de una cómoda que solo tiene 5 cajones: simplemente no está ahí.

1

La Regla de Oro

Si el arreglo tiene tamaño **N**, los índices van de **0** a **N-1**. Un arreglo de tamaño 5 tiene índices: 0, 1, 2, 3, 4. El índice 5 NO existe.

2

La Trampa del For

El error más común es usar `<=` en lugar de `<` en el bucle. Cuando la variable llega al tamaño del arreglo, intentará acceder a una posición inexistente.

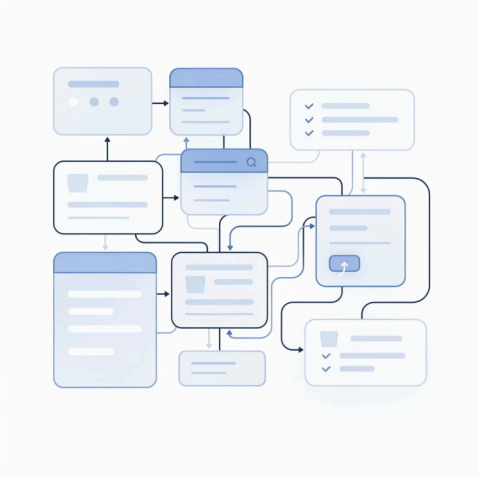
```
int[] vector = new int[5];

//
```

Modularización con Arrays

¿Por qué modularizar?

Para mantener el código limpio, nunca escriban el bucle for de impresión dentro del main. Creen herramientas reutilizables.

Lo interesante es que los Arreglos se pasan por referencia. Si modificás el arreglo dentro del método, ¡se modifica el original en el main!



```
public static void main(String[] args) {
    int[] misNotas = {10, 8, 9};
    mostrarVector(misNotas); // Llamada
limpia
}

// Método genérico para mostrar cualquier
vector
public static void mostrarVector(int[] v) {
    System.out.print("[ ");
    for (int i = 0; i < v.length; i++) {
        System.out.print(v[i] + " ");
    }
    System.out.println("]");
}
```

01

Declaración del Método

Creamos un método que recibe un arreglo de enteros como parámetro

02

Procesamiento Interno

El método trabaja con el arreglo usando su propio bucle y lógica

03

Reutilización

Podemos llamar este método desde cualquier parte con diferentes arreglos

Cierre del Módulo Lógico

Hemos completado el núcleo duro de la programación estructurada. Este recorrido nos ha llevado desde los conceptos más básicos hasta técnicas avanzadas de gestión de memoria y estructuras de datos. Cada concepto que aprendimos se construye sobre el anterior, formando una base sólida para lo que viene.

Condicionales

Controlamos el flujo de decisiones en nuestros programas

Ciclos

Automatizamos tareas repetitivas de forma eficiente

Arreglos

Manejamos colecciones de datos con valores por defecto

Stack Overflow

Entendemos los peligros de la recursividad sin límites

> "Con esta base sólida, estamos listos para avanzar hacia el paradigma de Objetos en la próxima unidad. El viaje recién comienza."